OPEN ROLE · DATA PLATFORM & APPLICATIONS

AI & Analytics Engineer

Microsoft Fabric platforms, and the applications and AI tooling that run on top of them. One role, two halves, both client-facing.

LOCATION	EXPERIENCE	FUNCTION	TYPE
Delhi / NCR	**3–5 years**	**Data platform & applications**	**Full-time**

01 About OpenBI

OpenBI is a data intelligence and analytics consultancy. OpenBI builds governed data platforms and the applications that sit on top of them for enterprise clients, largely in regulated industries.

The team is small, senior and AI-first. Every engineer here runs coding agents daily and ships production work with them. There is no layer of people between you and the problem.

02 The role

Half is data platform work on Microsoft Fabric. Most engagements start the same way: a client holds a sprawl of Power BI reports, each re-importing the same data and rebuilding the same logic. You replace that with one governed platform — ingest once, medallion lakehouse, a single certified Direct Lake semantic model, thin reports on top.

Half is application and AI engineering. Clients ask for the software around the data: internal portals, workflow tools, intelligence platforms, agent-driven automation. You build those end to end — backend, frontend, data layer, auth, deployment — and you own the performance and security work that keeps them standing.

You own outcomes on both sides. This is not a role where someone hands you a ticket.

03 What you'll do

DATA PLATFORM

- Build ingestion into Fabric from APIs, databases, flat files and document stores — full and incremental, orchestrated, monitored, restartable.
- Implement medallion architecture on Delta: Bronze as landed, Silver cleaned and deduplicated, Gold as a conformed star schema with SCD Type 2, surrogate keys and documented grain.
- Own the semantic layer end to end — Direct Lake model, DAX measures, relationships, row-level security — and defend every number in it during UAT.
- Rebuild reports against the governed model and stand up the app, alerts and scheduled distribution around it.
- Run data validation and reconciliation against the legacy estate, then document and hand over.

APPLICATIONS AND AI

- Build production web applications end to end: API design, data model, frontend, authentication and role-based access.
- Deploy and operate them on AWS — provisioning, networking, CI/CD, logging, monitoring, cost control.
- Own performance: query and index tuning, caching, pagination, payload size, and behaviour under concurrency.
- Own application security: authentication and session handling, authorization at the data layer, input validation, secrets management, dependency and configuration hygiene, and remediation when a review turns up findings.
- Build AI automation: agent orchestration, MCP servers, retrieval pipelines, evaluation harnesses.

04 What we need

- **3–5 years on a modern data platform.** Microsoft Fabric, or Azure data engineering (ADF with Databricks or Synapse, and PySpark) with a real intent to convert to Fabric.
- **Medallion architecture and dimensional modeling you can defend under questioning.** Bronze, Silver, Gold; star schema, SCD Type 2, surrogate keys, incremental patterns, grain discipline.
- **Power BI semantic modeling.** DAX measures, relationships, row-level security. You have owned a model, not only built visuals on someone else's.

04 What we need CONTINUED

- **End-to-end application delivery.** You have taken at least one application from an empty repository to live users — backend, frontend, database, auth, deployment — and kept it running afterwards.
- **Hands-on AWS deployment.** Compute, storage, managed databases, networking, IAM, CI/CD. You have deployed and debugged in production, not only followed a tutorial.
- **Application performance and security in practice.** You can describe a specific bottleneck you found and fixed, and a specific vulnerability you closed.
- **Python for ingestion and automation.** REST APIs, auth, pagination, retries, scheduling.
- **SQL and PySpark that hold up under a query plan**, not only under a small dataset.
- **Daily, evidenced use of coding agents** — Claude Code, Cursor, Codex or equivalent. Bring specifics: what you delegated, what you refused to delegate, and what you had to correct.
- **Git discipline.** Branches, pull requests, review.
- **Clear written English.** You will write documentation clients read.

05 Strong plus

- DP-600 or DP-700; Databricks Data Engineer Associate or Professional; AWS Solutions Architect or Developer Associate.
- Fabric depth: Dataflows Gen2, Data Activator, capacity behaviour, F-SKU sizing and throttling.
- RAG pipelines, MCP servers, multi-agent orchestration, prompt and output evaluation.
- React and TypeScript to a production standard, not a prototype standard.
- Infrastructure as code, containers, serverless.
- Both clouds — AWS and Azure — including enterprise identity and access.
- Delivery in regulated environments: validation, audit trails, controlled access.
- Migration experience: a cloud warehouse or on-premises estate into a lakehouse.

06 How we work

- **AI-first, review-gated.** Agents write a lot of our code. Nothing ships unreviewed. Your judgement over AI output matters more than your typing speed.
- **Direct client contact.** A senior leads the account; you are on the calls from week one.
- **Estimates in hours, and we hold to them.** Scope is written down before work starts.
- **A shared design system.** We do not freelance styling.
- **Strict contracts between components.** We favour orchestration we can debug over frameworks we cannot.

07 What this role is not

- Not a report-building seat. Producing visuals to a specification is a small fraction of the work.
- Not a research role. Everything you build goes into a client's production estate and stays there.
- Not agent theatre. We use AI where it compounds and write code by hand where it does not.

08 Interview process

01 **Screen.** Twenty minutes on background and motivation.

02 **Take-home.** Roughly four hours. A small multi-source dataset: build the medallion layers, a star schema, three non-trivial measures and one report. Submit with a short written note on where you used AI, where you deliberately did not, and what you had to correct.

03 **Technical deep-dive.** Sixty minutes on your submission, plus a walkthrough of an application you have shipped — architecture, deployment, and how you handled performance and security.

04 **Founder conversation.** Forty-five minutes.

A decision follows within five working days of the final round.

09 How to apply

The role is based in **Delhi / NCR**. Send your CV to **harry@openbi.ai**, with a link to something you have built if you have one.